Quiz a)

a) True
   - Tries to load address `0`

b) False
   - Computation on address

c) False
   - Move the value `0` to `%rcx`

d) False
   - Decreases the stack size

Quiz b)-e)

b)

- c) is the correct answer

c)

- a) Valid: `%ebx = 4*(%eax)`
- b) Invalid: 15 is a memory address not an immediate!
- c) Valid: store the content of `%eax` to address 655

d)

- a) is the correct answer

e)

```
mov %ebp, %esp
pop %ebp
```

Quiz f)

Systems@ETH Zürich

C Code

```
// input:  int x (in %edi)
// output: int y (in %eax)

1. int func(int x) {
2.    int y = 0;
3.    if (x > 0) {
4.       y = 10;
5.    }
6.    y += 5;
7.    return y;
8. }
```

Assembly (gcc 4.9.4 with -O0)

```
1. pushq %rbp
2. movq %rsp, %rbp
3. movl %edi, -20(%rbp)
4. movl $0, -4(%rbp)
5. cmpl $0, -20(%rbp)
6. jle .L2
7. movl $10, -4(%rbp)
.L2:
8. addl $5, -4(%rbp)
9. movl -4(%rbp), %eax
10.popq %rbp
11.ret
```

Quiz f)

Systems@ETH Zürich

C Code

```
// input:  int x (in %edi)
// output: int y (in %eax)

1. int func(int x) {
2.    int y = 0;
3.    if (x > 0) {
4.       y = 10;
5.    }
6.    y += 5;
7.    return y;
8. }
```

Assembly (gcc 4.9.4 with -O1)

```
1. testl %edi, %edi
2. movl $10, %edx
3. movl $0, %eax
4. cmovg %edx, %eax
5. addl $5, %eax
6. ret
```

Quiz g)

C Code

```
// input:  int x, int y
           (in %edi, %esi)
// output: int z (in %eax)
```

```
1. int func(int x, int y) {
2.    int z = 0;
3.    while (z <= y) {
4.       z += 3*(x+1);
5.    }
6.    return z;
7. }
```

Assembly (gcc 4.9.4 with -O0)

```
1.   pushq %rbp
2.   movq %rsp, %rbp
3.   movl %edi, -20(%rbp)
4.   movl %esi, -24(%rbp)
5.   movl $0, -4(%rbp)
6.   jmp .L2
.L3:
7.   movl -20(%rbp), %eax
8.   leal 1(%rax), %edx
9.   movl %edx, %eax
10.  addl %eax, %eax
11.  addl %edx, %eax
12.  addl %eax, -4(%rbp)
.L2:
13.  movl -4(%rbp), %eax
14.  cmpl -24(%rbp), %eax
15.  jle .L3
16.  movl -4(%rbp), %eax
17.  popq %rbp
18.  ret
```

Quiz g)

C Code

```
// input:  int x, int y
           (in %edi, %esi)
// output: int z (in %eax)
```

```
1. int func(int x, int y) {
2.   int z = 0;
3.   while (z <= y) {
4.     z += 3*(x+1);
5.   }
6.   return z;
7. }
```

Assembly (gcc 4.9.4 with -O1)

```
1. testl %esi, %esi
2. js .L4
3. leal 3(%rdi,%rdi,2), %edx
4. movl $0, %eax
.L3:
5. addl %edx, %eax
6. cmpl %eax, %esi
7. jge .L3
8. rep ret
.L4:
9. movl $0, %eax
10.ret
```

Quiz g)

Systems@ETH Zürich

C Code

```
// input:  int x, int y
           (in %edi, %esi)
// output: int z (in %eax)

1. int func(int x, int y) {
2.   int z = 0;
3.   while (z <= y) {
4.     z += 3*(x+1);
5.   }
6.   return z;
7. }
```

Assembly (gcc 4.9.4 with -Os)

```
1. incl %edi
2. xorl %eax, %eax
3. leal (%rdi,%rdi,2), %edi
.L2:
4. cmpl %esi, %eax
5. jg .L5
6. addl %edi, %eax
7. jmp .L2
.L5:
8. ret
```

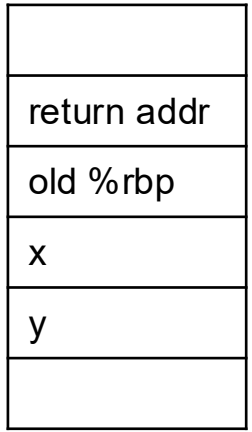
Quiz h)

Assembly

```
func(int, int):
1.  pushq %rbp
2.  movq %rsp, %rbp
3.  movl %edi, -4(%rbp)
4.  movl %esi, -8(%rbp)
5.  movl -4(%rbp), %eax
6.  cmpl -8(%rbp), %eax
7.  jle .L2
8.  movl -4(%rbp), %eax
9.  jmp .L3
.L2:
10. movl -8(%rbp), %eax
.L3:
11. popq %rbp
12. ret
```

C Code

```
1. int func(int x, int y) {
2.     if (x > y) {
3.         return x;
4.     } else {
5.         return y;
6.     }
7. }
```



`lea` is special...

Only `lea` operates on memory addresses, everything else on memory values

```
1. leaq 8(%rax, %rdx, 4), %rdi
2. addq 8(%rax, %rdx, 4), %rdi
```

addq will load the **value at that address**

Check your registers

Make sure instruction and register size match!

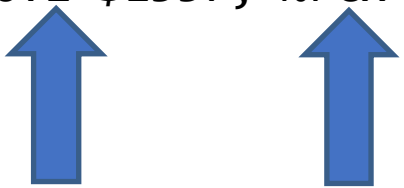
```
1. movl $1337, %rcx
```

```
1. asm.s: Assembler messages:
2. asm.s:28: Error: incorrect register `%rcx' used
with `l' suffix
```

Check your registers

Make sure instruction and register size match!

```
1. movl $1337, %rcx
```




```
1. asm.s: Assembler messages:
2. asm.s:28: Error: incorrect register `%rcx' used
with `l' suffix
```

objdump -d

- Disassambles object files to assembly

```
0000000000000000 <main>:
   0: 48 8d 04 bd 00 00 00 00 lea  0x0(, %rdi, 4), %rax
   8: c3                      retq
```

Side Note: `objdump -d`

- If you need wider/shorter lines, use

  no wrap: `objdump -d --wide`
  custom wrap: `objdump -d --insn-width=8`

Systems@ETH Zürich

Exam Tasks

Assembly: What is ARG?

Assembly: HS17-2-(0)

Systems@ETH Zürich

You have the following C Program:

```
#include <stdint.h>
int64_t multiply(int64_t a) {
  return a * ARG;
}
```

In the following disassemblies, what is ARG?

Assembly: HS17-2-(1)

```
int64_t multiply(int64_t a) {
   return a * ARG;
}
0000000000000000 <multiply>:
   0:  55              push %rbp
   1:  48 89 e5        mov  %rsp, %rbp
   4:  48 89 7d f8     mov  %rdi, -0x8(%rbp)
   8:  48 8b 45 f8     mov  -0x8(%rbp), %rax
   c:  48 c1 e0 02     shl  $0x2, %rax
  10:  5d              pop  %rbp
  11:  c3              retq
```

What is ARG?

Assembly: HS17-2-(1)

```
int64_t multiply(int64_t a) {
   return a * ARG;
}
0000000000000000 <multiply>:
   0:  push %rbp
   1:  mov  %rsp, %rbp
   4:  mov  %rdi, -0x8(%rbp)   # store a
   8:  mov  -0x8(%rbp), %rax   # load a => rax = a
   c:  shl  $0x2, %rax         # rax = 4 * a
  10:  pop  %rbp
  11:  retq
```

What is ARG?

ARG == 4

Assembly: HS17-2-(2)

```
int64_t multiply(int64_t a) {
  return a * ARG;
}
0000000000000000 <multiply>:
   0: 48 8d 04 bd 00 00 00 00 lea  0x0(, %rdi, 4), %rax
   8: c3                      retq
```

What is ARG?

Assembly: HS17-2-(2)

```
int64_t multiply(int64_t a) {
  return a * ARG;
}
0000000000000000 <multiply>:
   0: lea  0x0(, %rdi, 4), %rax   # rax = 0 + 4*a + 0
   8: retq
```

What is ARG?

ARG == 4

Assembly: HS17-2-(3)

```
int64_t multiply(int64_t a) {
  return a * ARG;
}
0000000000000000 <multiply>:
   0: 48 8d 04 7f  lea  0x0(%rdi, %rdi, 2), %rax
   4: 48 8d 04 87  lea  0x0(%rdi, %rax, 4), %rax
   8: c3           retq
```

What is ARG?

Assembly: HS17-2-(3)

```
int64_t multiply(int64_t a) {
  return a * ARG;
}
```

```
0000000000000000 <multiply>:
   0: lea  0x0(%rdi, %rdi, 2), %rax  # rax = a + 2*a
   4: lea  0x0(%rdi, %rax, 4), %rax  # rax = a + 4*(3a)
   8: retq
```

What is ARG?

ARG == 13

Assembly: HS17-2-(4)

```
int64_t multiply(int64_t a) {
  return a * ARG;
}
0000000000000000 <multiply>:
   0: 31 c0   xor  %eax, %eax
   2: c3      retq
```

What is ARG?

Assembly: HS17-2-(4)

```
int64_t multiply(int64_t a) {
  return a * ARG;
}
0000000000000000 <multiply>:
   0: xor  %eax, %eax    # eax = 0
   2: retq
```

What is ARG?

ARG == 0

Assembly: HS17-2-(5)

```
int64_t multiply(int64_t a) {
  return a * ARG;
}
0000000000000000 <multiply>:
   0: 48 8d 04 fd 00 00 00 00 lea  0x0(, %rdi, 8), %rax
   8: 48 29 f8                sub  %rdi, %rax
   b: 48 8d 04 c7             lea  (%rdi, %rax, 8), %rax
   f: c3                      retq
```

What is ARG?

Assembly: HS17-2-(5)

```
int64_t multiply(int64_t a) {
  return a * ARG;
}
```

```
0000000000000000 <multiply>:
   0: lea  0x0(, %rdi, 8), %rax   # rax = 8 * a
   8: sub  %rdi, %rax             # rax = 7a
   b: lea  (%rdi, %rax, 8), %rax  # rax = a + 8 * 7a
   f: retq
```

What is ARG?

ARG == 57

Assembly: HS17-2-(6)

```
int64_t multiply(int64_t a) {
  return a * ARG;
}
0000000000000000 <multiply>:
   0:  48 89 f8        mov  %rdi, %rax
   3:  48 f7 d8        neg  %rax
   6:  48 c1 e0 02     shl  $0x2, %rax
   a:  48 29 f8        sub  %rdi, %rax
   d:  48 c1 e0 02     shl  $0x2, %rax
  11:  c3              retq
```

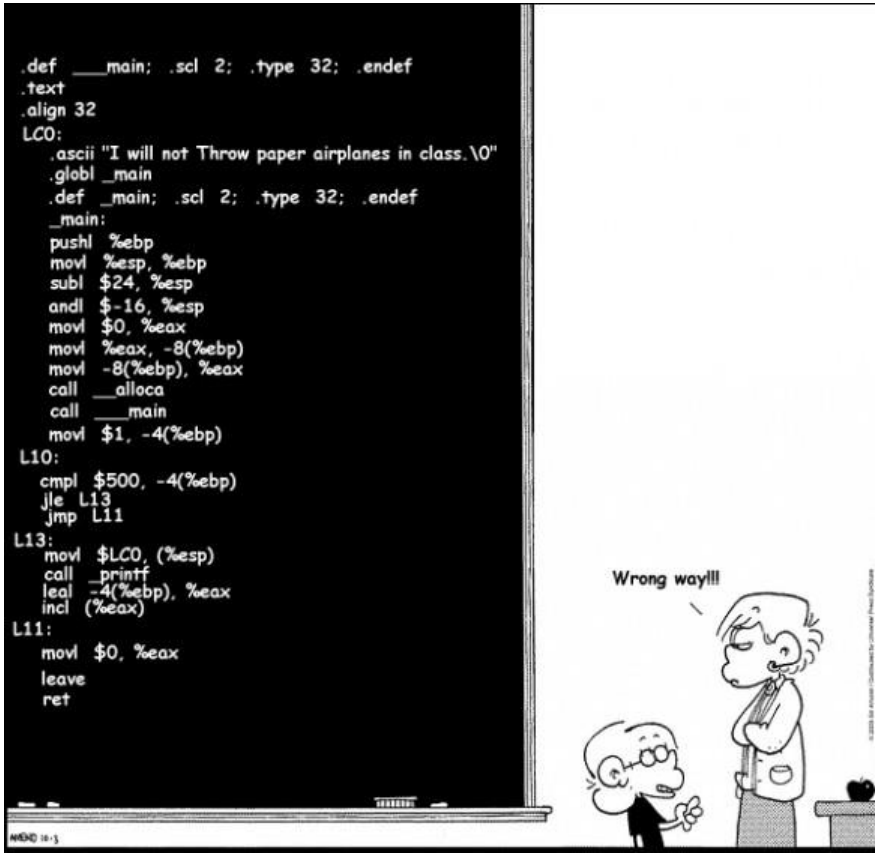
What is ARG?

Assembly: HS17-2-(6)

```
int64_t multiply(int64_t a) {
  return a * ARG;
}
0000000000000000 <multiply>:
   0:  mov  %rdi, %rax    # rax = a
   3:  neg  %rax          # rax = -a
   6:  shl  $0x2, %rax    # rax = -4a
   a:  sub  %rdi, %rax    # rax = -4a - a = -5a
   d:  shl  $0x2, %rax    # rax = 4 * (-5a) = -20a
  11:  retq
```

What is ARG?

ARG == -20



Enjoy the rest of your week!